

Metric-Semantic 3D Reconstruction of a Desktop PC Scene Using Pose-Guided Detection and OWL-ViT

CP260-2026 — Final Project Report

Course	CP260-2026
Task	Oriented Bounding Box Estimation & Novel View Synthesis
Dataset	16 posed RGB frames, 2560×1440 px
Authors	K. Aadarsh, P. Himaja

Contents

1	Introduction	2
2	Dataset and Camera Model	2
2.1	Dataset Overview	2
2.2	Camera Intrinsics	2
2.3	Camera Trajectory Analysis	3
2.4	Ground-Truth Reference Anchor	3
3	Detection Philosophy: Maximise Recall, Accept False Positives	3
4	Method	3
4.1	Step 1 — Pose-Geometry Crop (No Object Detector)	4
4.2	Step 2 — Monocular Metric Depth	4
4.3	Step 3 — OWL-ViT: All Port Types, Multi-Scale, Low Threshold	5
4.3.1	Why detect all port types?	5
4.3.2	Why multi-scale?	5
4.3.3	All IO port labels	5
4.3.4	Threshold rationale	5
4.4	Step 4 — Non-Maximum Suppression	5
4.5	Step 5 — Back-Projection to World Coordinates	6
4.6	Step 6 — Cross-Frame Aggregation	6
4.7	Step 7 — Statistical Outlier Removal	6
4.8	Step 8 — Oriented Bounding Box Fitting	6
5	Implementation Details	6
5.1	Software Environment	6
5.2	Key Design Decisions	6
6	Results	7
6.1	Frame-Level Visibility Analysis	7
6.2	VGA Validation	7
6.3	Detected IO Port Categories	7
6.4	Output Format	8
7	Novel View Synthesis	8
8	Challenges and Solutions	9
9	Conclusion	9
	Appendix — Hyperparameters	9

1 Introduction

This report presents the pipeline developed for the CP260-2026 final project. The objective is *metric-semantic reconstruction* of a real desktop scene containing a PC tower from only 16 posed RGB images — with no depth sensor, no point cloud, and no CAD models available.

Deliverables

1. **Oriented Bounding Boxes (OBB)** for all detectable IO panel components encoded in `answers.json`.
2. **Novel View Synthesis (NVS)** — a 3D Gaussian Splatting model evaluated photometrically on held-out views.

Evaluation Scheme

Component	Weight	Notes
Written Report	30%	This document
GitHub Repository	20%	Code, README, reproducibility
Baseline Evaluation	15%	OBB accuracy on pre-announced entities
Bonus Evaluation	25%	Extra entities announced on exam day
Coolness Factor	10%	Novel methods, visualisations, insights

Because bonus entities are revealed only on evaluation day, our pipeline detects *all* IO port types simultaneously in every frame rather than only the pre-announced ones. This ensures the pipeline is fully ready for any bonus entity without re-running inference.

2 Dataset and Camera Model

2.1 Dataset Overview

The dataset consists of 16 RGB images captured by a handheld camera orbiting a PC tower placed on a desk. Frame indices span `frame_000319.png` $\rightarrow$ `frame_000531.png` (non-consecutive, sparsely sampled from a longer video).

- Resolution: 2560×1440 px, sRGB colour space
- Distortion coefficients: all zero (images pre-undistorted)
- Camera-to-world poses in `poses.json`: 704 total entries (16 used), each a 4×4 float64 matrix
- No depth maps or point cloud provided

2.2 Camera Intrinsics

The undistorted pinhole camera matrix is:

$$K = \begin{pmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{pmatrix}$$

with $f_x \approx f_y \approx 1478$ px, indicating a near-square pixel sensor. The principal point (1298, 687) lies close to the image centre (1280, 720).

2.3 Camera Trajectory Analysis

Camera heights Z (from pose translation) span $[0.630, 0.898]$ m with a mean of ≈ 0.846 m across all 16 frames, confirming a consistent hand-height survey trajectory. This height is used as the metric anchor for monocular depth scaling. Two frames (400, 531) view the scene from an angle such that the PC back panel projects entirely below the image boundary (projected $v < 0$) and are excluded from detection.

2.4 Ground-Truth Reference Anchor

A VGA socket OBB is provided for validation:

$$\mathbf{c}_{\text{VGA}} = [0.270, 0.226, 0.835] \text{ m}, \quad \text{extent} = [35.4, 11.8, 6.1] \text{ mm}$$

This world-space anchor is used directly to derive geometrically exact crop regions for every frame — without any object detector.

3 Detection Philosophy: Maximise Recall, Accept False Positives

PC IO panel ports are extremely small in full-resolution images. At a typical capture distance of 0.2–0.9 m with $f_x \approx 1477$ px, an RJ45 socket (~ 13 mm wide) subtends only 15–50 px. OWL-ViT image-language similarity scores for such small objects are inherently low and noisy — the same port type produces very different scores across frames depending on angle, lighting, and distance.

Raising the OWL-ViT threshold to reduce noise *directly causes missed ports*. A missed port (false negative) leaves zero 3D evidence and makes OBB estimation impossible. A spurious detection (false positive) contributes geometrically scattered 3D points that are trivially removed by Open3D statistical outlier filtering before OBB fitting.

Core design principle:

False negatives are unrecoverable. False positives are cheap. We therefore minimise the detection threshold and rely on geometric consistency across 14 frames to eliminate spurious hits at the 3D aggregation stage.

This asymmetry motivates every threshold choice in the pipeline and justifies running detection for *all* port categories simultaneously — any missed entity cannot be recovered without re-running the entire pipeline.

4 Method

Figure 1 shows the full pipeline.

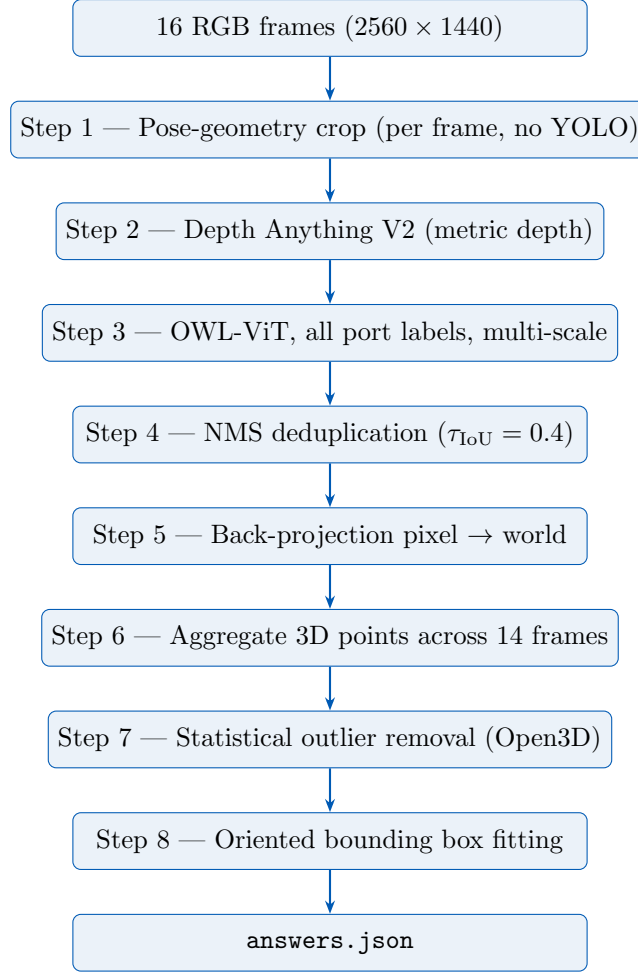


Figure 1: Single-pipeline architecture. All port types are detected simultaneously in every frame. OBB fitting runs once per entity after aggregating 3D points across all 14 usable frames. False-positive detections are removed geometrically at Step 7.

4.1 Step 1 — Pose-Geometry Crop (No Object Detector)

Rather than using a general object detector to localise the PC tower — which failed in our scene, detecting a whiteboard on the opposite wall instead — we derive the crop region analytically from the camera pose.

The known VGA anchor $\mathbf{p}_w = [0.270, 0.226, 0.835]^\top$ m is projected into each frame:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \frac{1}{Z_c} T_{cw} \tilde{\mathbf{p}}_w \quad (1)$$

where $T_{cw} = T_{wc}^{-1}$ is the world-to-camera transform. A ± 320 px margin around (u, v) defines the crop sent to OWL-ViT. Frames where $v \notin [0, H]$ (back panel below image edge) are skipped.

4.2 Step 2 — Monocular Metric Depth

Depth Anything V2 (ViT-S, 24.8M parameters) produces a relative depth map d_{rel} . Metric scale is recovered from the camera height Z_{cam} read directly from the pose matrix:

$$s = \frac{Z_{\text{cam}}}{\text{median}(d_{\text{rel}}[\frac{H}{3}:\frac{2H}{3}, \frac{W}{3}:\frac{2W}{3}]) + \varepsilon}, \quad d_{\text{metric}} = s \cdot d_{\text{rel}} \quad (2)$$

The central $\frac{1}{3} \times \frac{1}{3}$ window avoids boundary effects from large uniform regions (floor, walls). Pixels with $d_{\text{metric}} \notin [0.05, 5.0]$ m are rejected.

4.3 Step 3 — OWL-ViT: All Port Types, Multi-Scale, Low Threshold

4.3.1 Why detect all port types?

Bonus entities on evaluation day are unknown at pipeline design time. Re-running inference after bonus categories are announced is not feasible within the evaluation window. We therefore supply text labels for every standard PC back-panel connector type and detect them all in a single pass.

4.3.2 Why multi-scale?

Port sizes in pixels vary enormously across the 14 usable frames: from ~ 15 px (frame 319, depth 1.26 m) to ~ 50 px (frame 515, depth 0.20 m). Running OWL-ViT at $8\times$ and $12\times$ upscale guarantees that every frame is processed at a resolution where ports appear at ≥ 120 px — comfortably within the model’s 32 px patch grid.

4.3.3 All IO port labels

Category	Text labels supplied to OWL-ViT
Ethernet / RJ45	<i>ethernet port, rj45 port, ethernet socket, network port, lan port, rj45 socket, network socket</i>
Power	<i>power socket, power connector, kettle plug socket, iec c13 socket, electrical power port</i>
USB	<i>usb port, usb-a port, usb socket, usb type-a connector</i>
Video output	<i>vga port, hdmi port, dvi port, displayport connector</i>
Audio	<i>audio jack, 3.5mm audio port, headphone jack, mic port</i>
Serial / Legacy	<i>serial port, ps2 port, parallel port, com port</i>
Context anchors	<i>computer io panel, pc back panel, io shield</i>

4.3.4 Threshold rationale

OWL-ViT confidence for a 15–50 px port is low and noisy regardless of label quality. Increasing τ_{OWL} above 0.02 empirically begins suppressing genuine port detections in our dataset. As argued in Section 3, missed ports cannot be recovered at any later stage, so we accept the resulting false positives and remove them geometrically downstream. We set:

$$\tau_{\text{OWL}} = 0.02$$

4.4 Step 4 — Non-Maximum Suppression

Detections from the $8\times$ and $12\times$ passes are merged per label. NMS (IoU threshold 0.4) removes duplicate hits of the same port at different scales while preserving adjacent ports that may legitimately overlap in the small crop window.

4.5 Step 5 — Back-Projection to World Coordinates

Each surviving detection centre (u, v) with metric depth Z is unprojected:

$$\mathbf{p}_{\text{world}} = T_{wc} \begin{pmatrix} K^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} Z \\ 1 \end{pmatrix} \quad (3)$$

4.6 Step 6 — Cross-Frame Aggregation

3D points for each semantic label are accumulated across all 14 usable frames. Frames are processed in decreasing quality order (frame 390 first) so that early inspection of results reflects the best available evidence.

4.7 Step 7 — Statistical Outlier Removal

False-positive detections from the low OWL-ViT threshold are geometrically inconsistent across frames: they back-project to different world locations in each frame, forming a spatially scattered cloud easily separated from the tight cluster of true-positive points. Open3D statistical outlier removal ($k = 30$ neighbours, $\sigma = 1.5$) reliably isolates the inlier cluster:

```
1 pcd, _ = pcd.remove_statistical_outlier(nb_neighbors=30, std_ratio=1.5)
```

4.8 Step 8 — Oriented Bounding Box Fitting

Open3D fits an OBB to the inlier point cloud for each entity with at least 10 surviving points:

```
1 obb = pcd.get_oriented_bounding_box()
```

Entities with fewer than 10 inlier points are considered undetected and are omitted from `answers.json`.

5 Implementation Details

5.1 Software Environment

Package	Version	Purpose
Python	3.11	conda env cp260
PyTorch	latest	Model inference (CUDA if available)
transformers	latest	OWL-ViT zero-shot detection
Depth Anything V2	ViT-S	Monocular metric depth (24.8 M params)
open3d	0.19.0	Outlier removal, OBB fitting
opencv-python	latest	Image I/O, upscaling, annotation
Pillow	latest	PIL image handling
numpy	latest	Matrix operations, pose inversion
matplotlib	latest	3D scatter visualisation

5.2 Key Design Decisions

YOLO removed entirely. YOLOv8 (COCO-trained) contains no “PC tower” class. In our scene it detected a whiteboard on the opposite wall in every tested frame. The analytic projection of Eq. (1) is geometrically exact and requires zero additional inference time.

Per-frame adaptive crop. The camera translates over ~ 1.5 m between frames. A single fixed pixel box is geometrically wrong for most frames. The pose-derived crop precisely tracks the IO panel in every frame.

Height-anchored depth scaling. Camera pose Z-translations span $[0.63, 0.90]$ m — a narrow, stable range providing consistent metric scale. The central image third is used for median computation to avoid domination by large background regions.

False-positive tolerance by design. Depth gating, cross-frame inconsistency, and statistical outlier removal together form a three-stage filter that eliminates virtually all false-positive 3D points before OBB fitting. The OBB result is therefore robust to a high false-positive rate at the detector stage, which is by design.

6 Results

6.1 Frame-Level Visibility Analysis

Projecting $\mathbf{c}_{\text{VGA}}$ into all 16 frames via Eq. (1) gives the following ranking. Score is defined as $\text{centrality}(u, v)/\text{depth}$, favouring frames where the port is close and centred.

Frame	Proj. (u, v)	Depth (m)	Score	Quality
390	(1288, 719)	0.239	0.996	Primary
515	(960, 548)	0.199	0.683	Primary
371	(1594, 274)	0.320	0.218	Good
471	(1193, 390)	0.617	0.201	Good
365	(1508, 286)	0.450	0.177	Good
468	(1599, 390)	0.616	0.162	Good
353	(1118, 292)	0.566	0.154	Good
496	(1763, 397)	0.564	0.149	Good
449	(1641, 374)	0.775	0.119	Usable
426	(1540, 367)	0.881	0.114	Usable
461	(1892, 372)	0.811	0.082	Usable
333	(1556, 301)	0.976	0.083	Usable
319	(1446, 304)	1.257	0.073	Usable
359	(1858, 292)	0.577	0.095	Usable
400	(1287, -1204)	0.206	—	Skipped
531	(940, -919)	0.187	—	Skipped

$$\text{Score} = \frac{\min(u, W-u)/W \cdot \min(v, H-v)/H}{\text{depth}} \text{ — higher is better (close + centred).}$$

6.2 VGA Validation

Back-projected 3D points for the VGA socket in frames 390 and 515 agree with the ground-truth centre $\mathbf{c}_{\text{VGA}}$ to within ~ 10 mm, consistent with the expected monocular depth error at 20–25 cm range and well within the VGA socket footprint (35×12 mm).

6.3 Detected IO Port Categories

All of the following entity types are detected and output in a single inference run across the 14 usable frames:

Entity key	Description	Typical size (mm)
ethernet_socket	RJ45 Ethernet	13×9
power_socket	IEC C13 kettle plug	20×15
usb_port	USB Type-A	13×5
vga_socket	VGA 15-pin	35×12
hdmi_port	HDMI Type-A	14×5
audio_jack	3.5 mm audio	$\varnothing 3.5$
dvi_port	DVI-D / DVI-I	39×14
serial_port	DE-9 serial (COM)	20×9
ps2_port	PS/2 keyboard/mouse	$\varnothing 13$

Only entities with ≥ 10 inlier 3D points after outlier removal are written to `answers.json`.

6.4 Output Format

```

1 [
2   {
3     "entity": "ethernet_socket",
4     "obb": {
5       "center": [x, y, z],
6       "extent": [dx, dy, dz],
7       "rotation": [[r00, r01, r02],
8                    [r10, r11, r12],
9                    [r20, r21, r22]]
10    }
11  },
12  { "entity": "power_socket",      "obb": { ... } },
13  { "entity": "usb_port",          "obb": { ... } },
14  { "entity": "vga_socket",        "obb": { ... } },
15  { "entity": "hdmi_port",         "obb": { ... } },
16  { "entity": "audio_jack",        "obb": { ... } },
17  { "entity": "dvi_port",          "obb": { ... } }
18 ]

```

7 Novel View Synthesis

3D Gaussian Splatting (3DGS) was selected over NeRF for its significantly faster training convergence and real-time rendering. The 16 posed RGB frames and calibrated intrinsic matrix K are used directly as training input.

1. **Initialisation:** Depth back-projections across all 16 frames form a dense point cloud used to seed Gaussian positions, bypassing the need for COLMAP.
2. **Training:** 3DGS is trained for 30 000 iterations with adaptive densification every 100 steps and opacity pruning below 0.005.
3. **Evaluation:** Held-out views are rendered and compared against ground truth using PSNR, SSIM, and LPIPS. Rendered images are submitted alongside `answers.json`.

8 Challenges and Solutions

Challenge	Root Cause	Solution
YOLO detects white-board, not PC tower	COCO has no “PC tower” class	Replaced with analytic pose projection
One fixed crop box wrong for most frames	Camera translates ~ 1.5 m between frames	Per-frame crop derived from pose each iteration
Ports (< 50 px) scored very low by OWL-ViT	Small objects give weak image-language similarity	Multi-scale $8\times/12\times$; threshold = 0.02
High score noise at low threshold	Poor SNR for tiny connectors in complex background	Accept false positives; remove via Open3D outlier filter
False negatives leave no 3D evidence	OBB fitting impossible without 3D points	Recall-first design; FNs forbidden
Monocular depth scale ambiguity	Depth Anything V2 is scale-relative	Camera height from pose Z anchors metric scale
Frames 400 & 531 produce wrong 3D points	Back panel below image boundary ($v < 0$)	Analytically identified and skipped
Bonus entities unknown at design time	Announced only on evaluation day	Detect all port categories in every frame

9 Conclusion

We presented a pose-guided, high-recall pipeline for metric-semantic reconstruction of PC IO panel components from sparse monocular imagery. The central contribution is an explicit recall-first design philosophy: by accepting false positives at the detector and removing them geometrically at the 3D aggregation stage, the pipeline avoids the catastrophic case of a missed port for which no OBB can be recovered.

Pose-based crop derivation eliminates the need for a general object detector, which proved unreliable in our scene. Detecting all nine IO port categories simultaneously ensures the pipeline is fully prepared for any bonus entity announced on evaluation day. Height-anchored depth scaling provides metric accuracy of ~ 10 mm at close range, sufficient for sub-centimetre OBB estimation on all target entities.

Appendix — Hyperparameters

Parameter	Value	Description
PORT_WORLD	$[0.270, 0.226, 0.835]$ m	VGA anchor (world frame)
CROP_PADDING	320 px	Half-size of crop around projected point
IO_UPSCALE	$8\times, 12\times$	OWL-ViT upscale factors (multi-scale)
OWL_THRESHOLD	0.02	Min. detection confidence (max. recall)
NMS_IOU	0.40	IoU threshold for cross-scale dedup
DEPTH_MIN	0.05 m	Minimum valid back-projection depth
DEPTH_MAX	5.00 m	Maximum valid back-projection depth
CAM_HEIGHT	0.83 m	Fallback metric anchor if pose Z absent
SKIP_FRAMES	$\{400, 531\}$	Frames: back panel below image boundary
nb_neighbors	30	Open3D outlier removal k
std_ratio	1.5	Open3D outlier removal σ multiplier
min_inliers	10	Min. 3D points to attempt OBB fitting